

1 系统实现

下面简要阐述 `loopSubdivision` 与 `root3Subdivision` 的实现逻辑。

1.1 Loop 细分实现

Loop 细分算法的实现严格遵循“先顶点更新，后拓扑分裂”的流程，在 `loopSubdivision.h` 中实现。

1. **偶顶点位置更新**：首先遍历所有旧顶点（偶顶点）。对于内部顶点，根据其度数 n 计算权重 β ，利用公式 $v_{new} = (1 - n\beta)v + \beta \sum v_{neighbor}$ 进行平滑。

```
1 // 内部顶点的加权平滑
2 computeWeightedPosition(neighbors, beta(v->valence), v->child->p);
```

2. **奇顶点生成**：遍历所有边生成新顶点（奇顶点）。对于内部边，新顶点位置由边的两个端点（权重 $3/8$ ）和共享该边的两个对角顶点（权重 $1/8$ ）决定。

```
1 // 内部边的奇顶点生成 (v2, v3为对角顶点)
2 odd->p = (e.verts[0]->p + e.verts[1]->p) * 0.375f + (v2->p + v3->p) * 0.125f;
```

3. **拓扑重构 (1 拆 4)**：在计算完所有新旧顶点位置后，进行网格连接关系的更新。对于每个原始三角形 (v_0, v_1, v_2) 及其三条边上的新顶点 (e_0, e_1, e_2) ，我们将其拆分为四个新的小三角形。

```
1 // 1个大三角形拆分为4个小三角形
2 f->child[0]->verts = {v0, e0, e2}; // 角部1
3 f->child[1]->verts = {e0, v1, e1}; // 角部2
4 f->child[2]->verts = {e2, e1, v2}; // 角部3
5 f->child[3]->verts = {e0, e1, e2}; // 中心
```

1.2 $\sqrt{3}$ 细分实现

$\sqrt{3}$ 细分采用了一种基于面重心插入与边翻转的特殊策略，代码位于 `root3Subdivision.h`。

1. **重心插入**：遍历当前网格的所有面，计算每个面的几何中心（Centroid），并将其作为一个新顶点插入到点集中。
2. **顶点平滑**：更新原始顶点的坐标。对于每个顶点，计算其所有一环邻域顶点的平均位置 $\bar{u}$ ，然后根据公式 $v_{new} = (1 - \alpha_n)v + \alpha_n\bar{u}$ 进行混合。
3. **基于边翻转的重三角化**：这是实现中的难点。我们需要遍历每一条原始边，如果是内部边，则连接该边两侧的重心点，并与原边的端点构成新的三角形。

```
1 if (e.boundary()) {
2     // 边界边：直接连接到唯一的面重心
3     newFaces.push_back(new SDFace(v_start, v_end, c0));
4 } else {
5     // 内部边：连接两侧重心点 c0, c1 与端点，产生两个新面（边翻转）
```

```

6      SDVertex *c1 = e.fs[1]->sdverts[0];
7      newFaces.push_back(new SDFace(v_start, c1, c0));
8      newFaces.push_back(new SDFace(v_end, c0, c1));
9  }

```

2 实验结果与分析

2.1 网格细分效果对比

细分算法的核心目标是“由粗变滑”。我们以 Glass 模型为例，对比了 Loop 细分与 $\sqrt{3}$ 细分在视觉和拓扑上的差异。

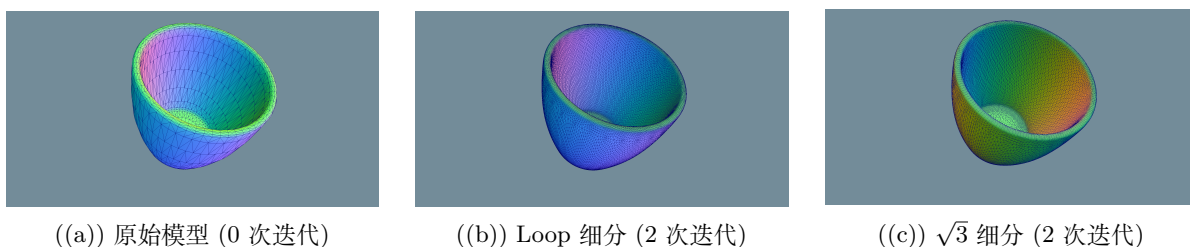


Figure 1: 不同细分算法效果对比。注意观察：Loop 细分产生的网格较为规则致密，而 $\sqrt{3}$ 细分呈现特殊的六边形对偶结构。

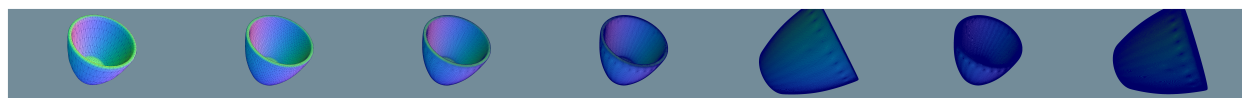


Figure 2: Loop 细分效果



Figure 3: Root 细分效果

从视觉效果上看（上图），两种算法均显著提升了模型表面的光顺度，原本生硬的棱角被平滑曲面取代。仔细观察网格线的走向可以发现：

- **Loop 细分**：网格密度增长极快（每次迭代面片数 $\times 4$ ），生成的三角形分布非常均匀，仅在原始奇异点附近保留了特异性。
- **$\sqrt{3}$ 细分**：面片数增长相对温和（每次迭代面片数 $\times 3$ ），网格拓扑呈现出更有机的流动感，这得益于其独特的重心插入与边翻转机制。

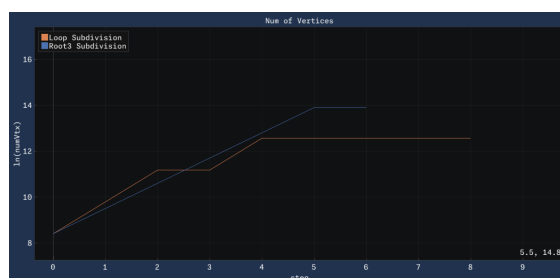
2.2 算法性能分析

为了量化比较不同算法的复杂度增长趋势，我们记录了算法运行时的顶点数量和面片数量，并绘制了其的对数增长曲线（如下图所示）。

从统计图中可以清晰地观察到：



(a) 面片数量



(b) 顶点数量

Figure 4: Loop 细分与 $\sqrt{3}$ 细分的几何增长趋势

- **指数增长特性：**无论是 Loop 细分还是 $\sqrt{3}$ 细分，其面片数和顶点数均呈现线性增长（在半对数坐标系下），这意味着几何复杂度随迭代次数呈指数级上升。
- **增长率差异：**Loop 细分的直线斜率明显高于 $\sqrt{3}$ 细分。这是符合理论预期的：Loop 细分每次迭代几何量乘以 4，而 $\sqrt{3}$ 细分乘以 3。因此，在达到相同的平滑度要求时， $\sqrt{3}$ 细分能够生成更轻量级的网格，这对显存受限的实时渲染应用更为友好。

3 代码

实验代码已经上传 github https://github.com/LoveGump/nku_CG_geometry-algs